Rapport de TP

Analyse statique d'une APK Android

Application étudiée : UnCrackable Level 1 (OWASP MAS)

Réalisé par : Amira Ezbiri

Module : Sécurité / Analyse d'APK Android

Outils utilisés : JADX GUI, dex2jar, JD-GUI

Type d'analyse : Analyse statique

Année universitaire 2025–2026

Table des matières

1 Introduction **2**

2 Préparation du workspace **2**

3 Téléchargement de l'APK **3**

4 Analyse avec JADX GUI **3**

4.1 Observations sur le manifeste 4

4.2 Observations sur les chaînes visibles 4

5 Recherche de chaînes sensibles **4**

6 Conversion DEX vers JAR **7**

7 Comparaison entre JADX GUI et JD-GUI **8**

8 Conclusion **8**

9 Annexe – Synthèse rapide **8**

1 Introduction

Dans ce TP, j'ai réalisé une analyse statique de l'APK *UnCrackable Level 1*. L'objectif était d'examiner la structure de l'application, d'observer son manifeste, de rechercher des chaînes sensibles dans le code décompilé et de comparer les outils de décompilation utilisés pendant le TP.

J'ai travaillé uniquement à partir de l'APK et de sa décompilation, sans exécuter l'application. Les captures d'écran insérées dans ce rapport montrent les différentes étapes que j'ai suivies.

2 Préparation du workspace

J'ai commencé par créer un dossier de travail, puis j'ai vérifié que le fichier téléchargé était bien une archive APK valide. J'ai aussi listé le contenu de l'archive et calculé son empreinte SHA-256 pour garder une trace du fichier analysé.

```
Sélection Administrateur : Windows PowerShell
Windows PowerShell
Copyright (C) Microsoft Corporation. Tous droits réservés.

Installez la dernière version de PowerShell pour de nouvelles fonctionnalités et améliorations ! https://aka.ms/PSWindow
s

PS C:\WINDOWS\system32> $WorkingDir = "$env:USERPROFILE\android-audit"
PS C:\WINDOWS\system32> New-Item -ItemType Directory -Force -Path $WorkingDir | Out-Null
PS C:\WINDOWS\system32> Set-Location $WorkingDir
PS C:\Users\HP\android-audit> $ApkUrl = "https://github.com/OWASP/owasp-mastg/raw/master/Crackmes/Android/Level_01/UnCra
ckable-Level1.apk"
PS C:\Users\HP\android-audit> $ApkDest = "$WorkingDir\UnCrackable-Level1.apk"
PS C:\Users\HP\android-audit> Invoke-WebRequest -Uri $ApkUrl -OutFile $ApkDest
PS C:\Users\HP\android-audit> Write-Host "Téléchargé : $ApkDest"
Téléchargé : C:\Users\HP\android-audit\UnCrackable-Level1.apk
PS C:\Users\HP\android-audit> Write-Host "Taille : $((Get-Item $ApkDest).Length / 1KB) KB"
Taille : 65.0888671875 KB
```

FIGURE 1 – Préparation de l'espace de travail et téléchargement de l'APK.

```
PS C:\Users\HP\android-audit> $RawBytes = [System.IO.File]::ReadAllBytes("C:\Users\HP\android-audit\UnCrackable-Level1.a
pk")
PS C:\Users\HP\android-audit> $RawBytes[0..3] | ForEach-Object { "{0:X2}" -f $_ }
50
4B
03
04
PS C:\Users\HP\android-audit>
```

FIGURE 2 – Vérification des premiers octets de l'archive APK : `50 4B`, ce qui confirme le format ZIP/APK.

```
PS C:\Users\HP\android-audit> Add-Type -AssemblyName System.IO.Compression.FileSystem
PS C:\Users\HP\android-audit> $Archive = [System.IO.Compression.ZipFile]::OpenRead((Resolve-Path "C:\Users\HP\android-au
dit\UnCrackable-Level1.apk").Path)
PS C:\Users\HP\android-audit> $Archive.Entries | Select-Object -ExpandProperty FullName -First 20
AndroidManifest.xml
META-INF/CERT.RSA
META-INF/CERT.SF
META-INF/MANIFEST.MF
classes.dex
res/layout/activity_main.xml
res/menu/menu_main.xml
res/mipmap-hdpi-v4/ic_launcher.png
res/mipmap-mdpi-v4/ic_launcher.png
res/mipmap-xhdpi-v4/ic_launcher.png
res/mipmap-xxhdpi-v4/ic_launcher.png
res/mipmap-xxxhdpi-v4/ic_launcher.png
resources.arsc
PS C:\Users\HP\android-audit> $Archive.Dispose()
PS C:\Users\HP\android-audit>
```

FIGURE 3 – Liste partielle du contenu de l'archive.

```
PS C:\Users\HP\android-audit> $HashResult = Get-FileHash -Algorithm SHA256 "C:\Users\HP\android-audit\UnCrackable-Level1
.apk"
PS C:\Users\HP\android-audit> $HashResult | Format-List | Out-File "sha256_apk.txt"
PS C:\Users\HP\android-audit> $HashResult

Algorithm       Hash                                                                   Path
---------       ----                                                                   ----
SHA256          1DA8BF57D266109F9A07C01BF7111A1975CE01F190B9D914BCD3AE3DBEF96F21       C:\Users\HP\android-audit\UnC...


PS C:\Users\HP\android-audit>
```

FIGURE 4 – Calcul du hash SHA-256 de l'APK pour assurer la traçabilité.

D'après mes vérifications, l'archive contient notamment `AndroidManifest.xml`, `classes.dex`, `resources.arsc` et les ressources de l'application.

3 Téléchargement de l'APK

J'ai récupéré l'APK *UnCrackable Level 1* depuis la plateforme OWASP MAS, qui propose des exercices pédagogiques d'analyse d'applications Android. Le fichier a ensuite été placé dans mon dossier de travail pour être étudié localement.

La taille du fichier affichée dans mon terminal est d'environ 65 Ko, ce qui correspond à une application volontairement légère pour l'exercice.

4 Analyse avec JADX GUI

J'ai ouvert l'APK dans JADX GUI afin d'explorer le manifeste et les ressources. Cette étape m'a permis d'identifier rapidement les paramètres principaux de l'application.

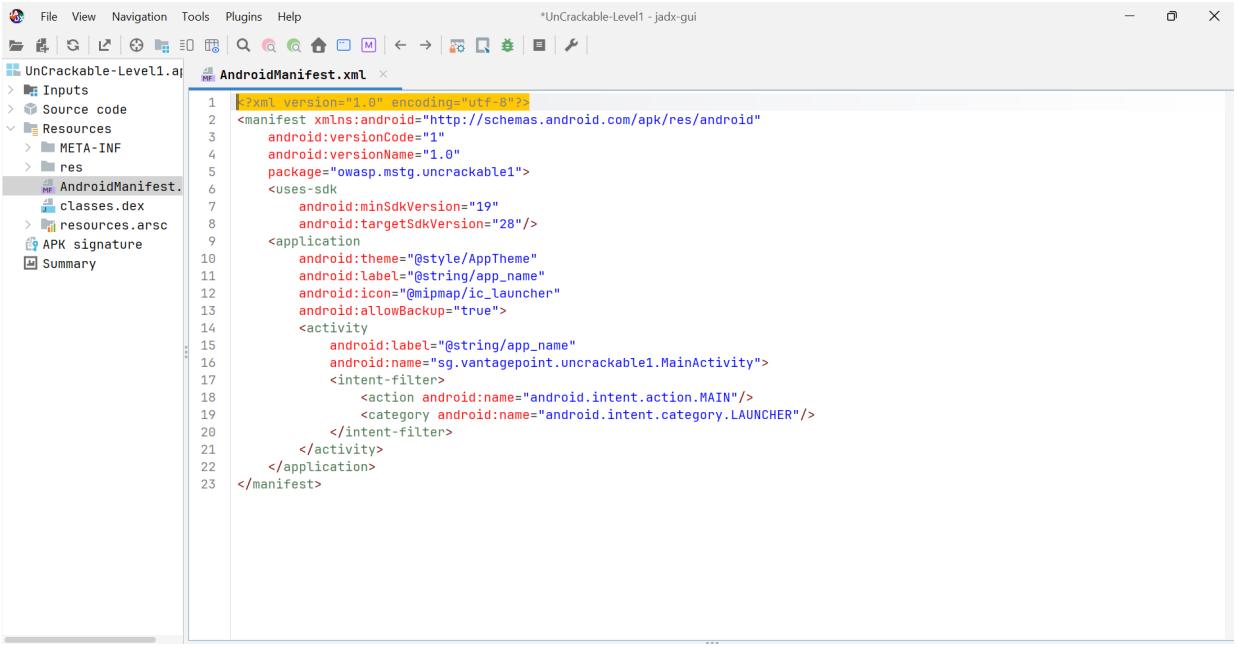

FIGURE 5 – Analyse du fichier `AndroidManifest.xml` dans JADX GUI.

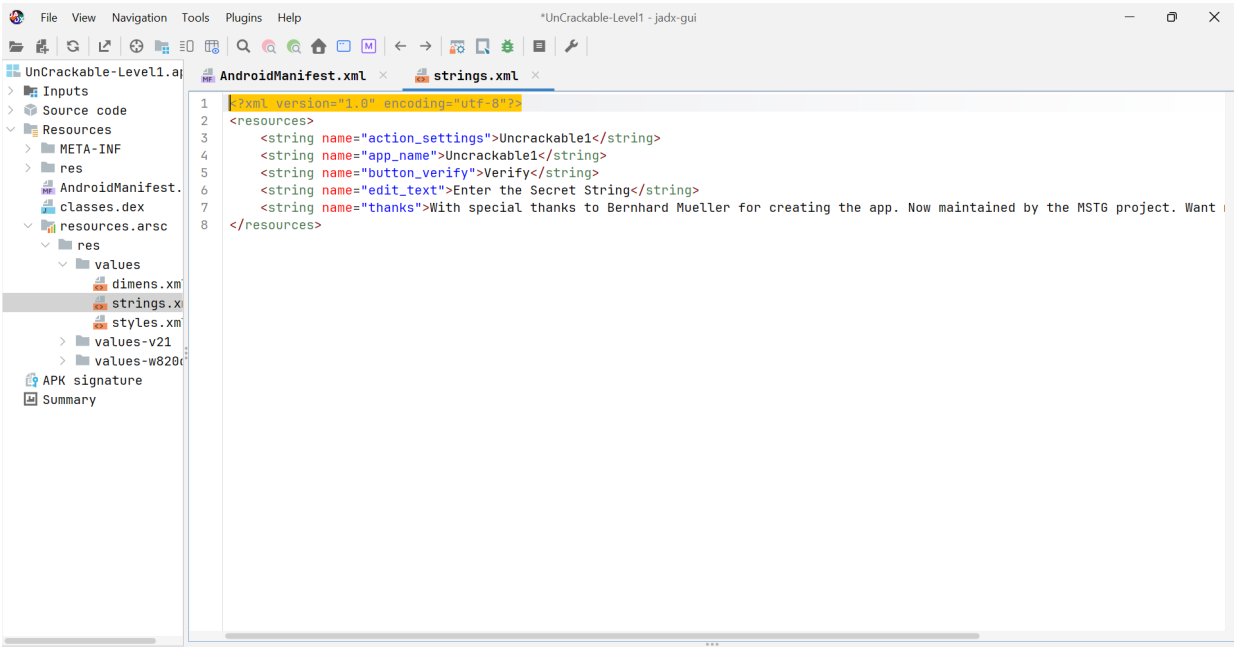

FIGURE 6 – Analyse du fichier `strings.xml` dans JADX GUI.

4.1 Observations sur le manifeste

À partir du manifeste, j'ai relevé les informations suivantes :

Package	`owasp.mstg.uncrackable1`
Version code	`1`
Version name	`1.0`
Version SDK minimale	`19`
Version SDK cible	`28`
Attribut notable	`android:allowBackup="true"`
Activity principale	`sg.vantagepoint.uncrackable1.MainActivity`

Le point qui m'a particulièrement marqué est la présence de `allowBackup="true"`, car cela peut faciliter l'extraction de certaines données si l'application n'est pas protégée correctement.

4.2 Observations sur les chaînes visibles

Dans `strings.xml`, j'ai trouvé plusieurs chaînes simples et explicites comme `Uncrackable1`, `Verify` et `Enter the Secret String`. Ces chaînes confirment qu'il s'agit d'une application d'entraînement conçue pour l'analyse de sécurité.

5 Recherche de chaînes sensibles

J'ai utilisé la fonction de recherche globale de JADX pour identifier des mots-clés intéressants dans le code. Cette étape m'a permis de repérer plusieurs éléments liés à la sécurité et au comportement interne de l'application.

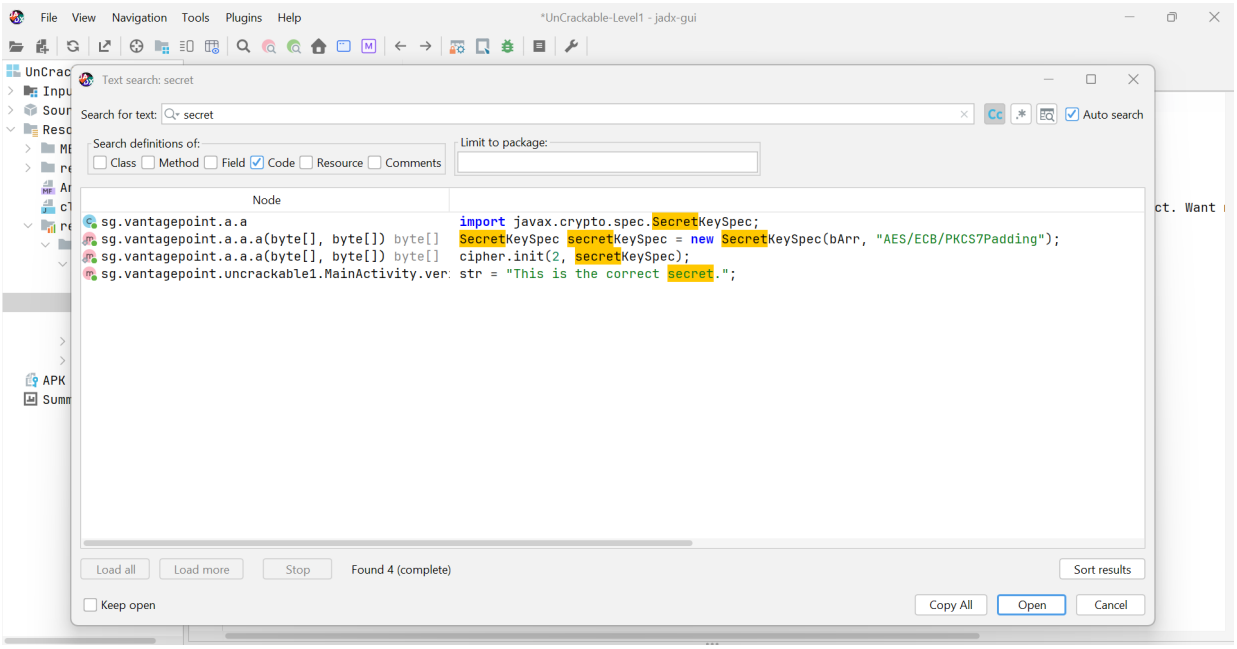

FIGURE 7 – Résultats de recherche sur le mot-clé `secret`.

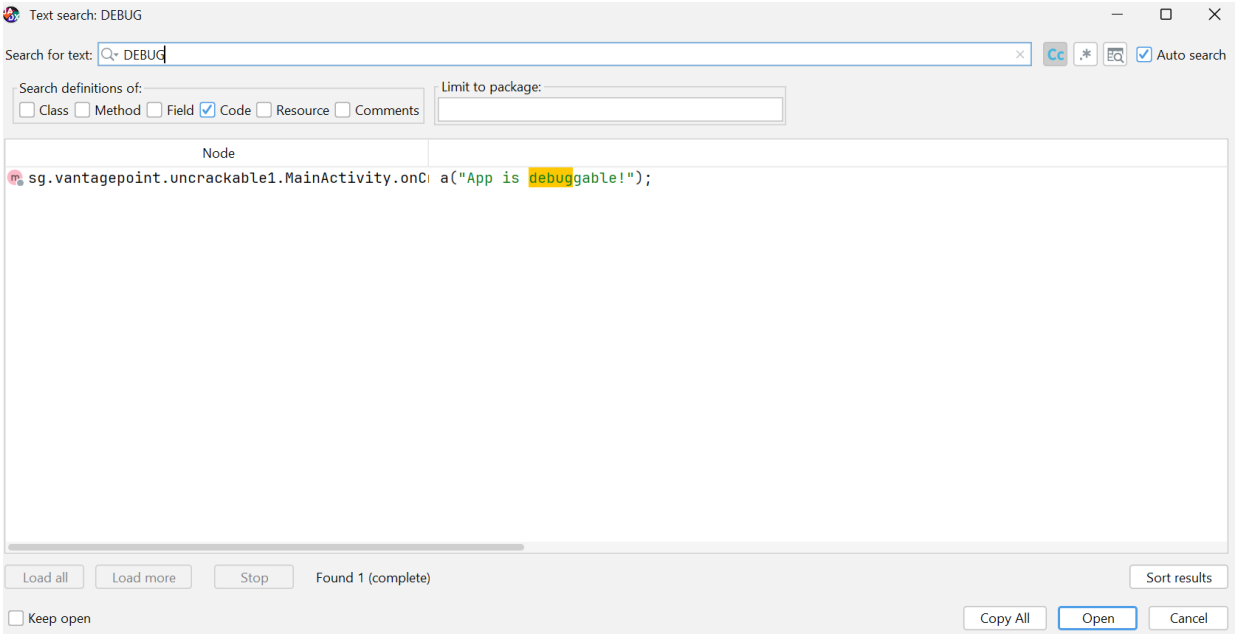

FIGURE 8 – Résultat de recherche sur `DEBUG`.

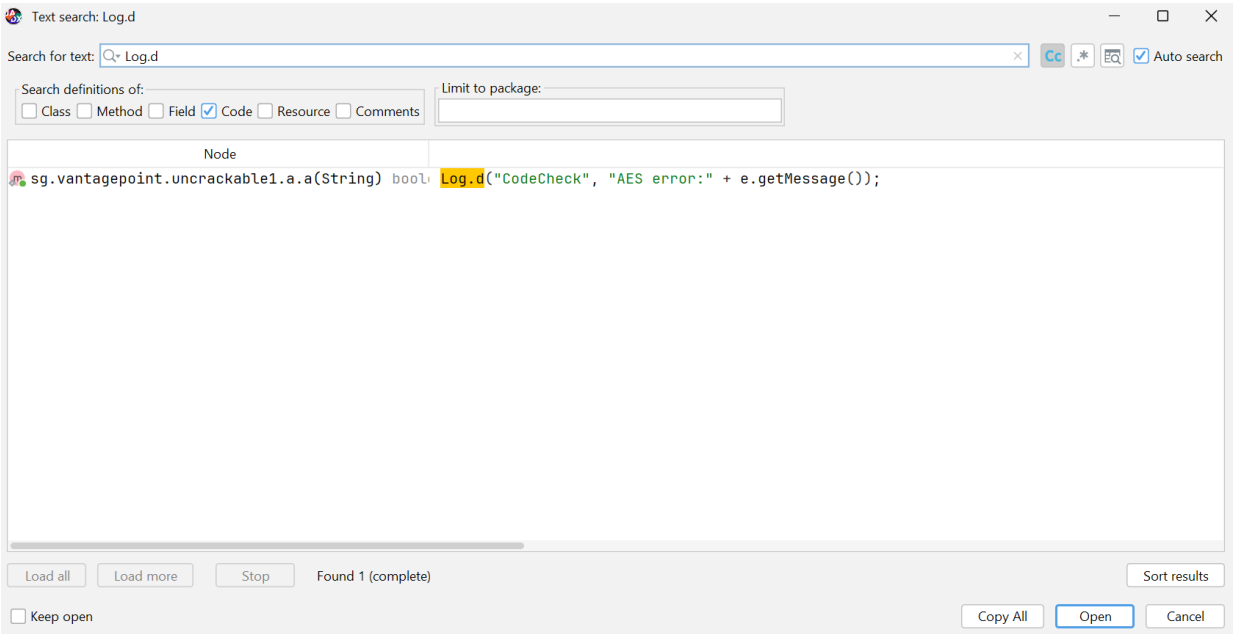

FIGURE 9 – Résultat de recherche sur `Log.d`.

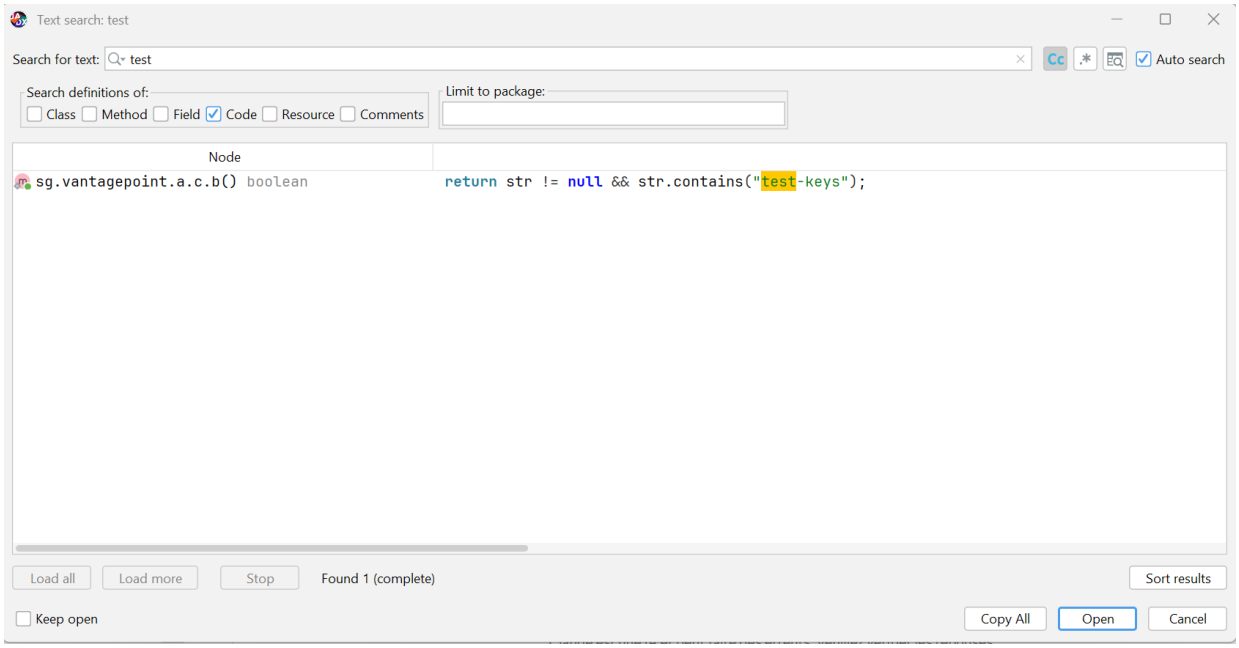

FIGURE 10 – Résultat de recherche sur `test`.

Mot-clé	Observation	Impact
`secret`	Présence de `SecretKeySpec`, de l'algorithme `AES/ECB/PKCS7Padding` et de la chaîne `This is the correct secret.`	**Le secret et le mécanisme de vérification sont visibles dans le code décompilé.**
`DEBUG`	Message `App is debuggable!` trouvé dans `MainActivity`	**L'application affiche un indice de débogage dans son code.**
`Log.d`	Journalisation avec `CodeCheck` et `"AES error:" + e.getMessage()`	**Des informations techniques peuvent apparaître dans les logs.**
`test`	Test avec `str.contains("test-keys")`	**Présence d'une vérification liée à l'environnement ou au root.**

À mon niveau d'analyse, le secret le plus important est clairement visible dans le bytecode décompilé, ce qui montre qu'une logique sensible ne doit pas être laissée en clair dans une application.

6 Conversion DEX vers JAR

J'ai ensuite extrait le fichier `classes.dex` de l'APK et je l'ai converti en fichier JAR à l'aide de *dex2jar*. Cette étape m'a servi à comparer la lisibilité du code avec l'outil JD-GUI.

```
PS C:\Users\HP\android-audit> $ExtractionDir = "$WorkingDir\dex-extracted"
PS C:\Users\HP\android-audit> New-Item -ItemType Directory -Force -Path $ExtractionDir | Out-Null
PS C:\Users\HP\android-audit> $ApkArchive = [System.IO.Compression.ZipFile]::OpenRead((Resolve-Path "C:\Users\HP\android
-audit\UnCrackable-Level1.apk").Path)
PS C:\Users\HP\android-audit> $ApkArchive.Entries | Where-Object { $_.Name -match "^classes.*\.dex$" } | ForEach-Object
{ $TargetPath = Join-Path $ExtractionDir $_.Name     [System.IO.Compression.ZipFileExtensions]::ExtractToFile($_, $Targe
tPath, $true)  Write-Host "Extrait : $($_.Name)"}
Join-Path : Impossible de trouver un paramètre positionnel acceptant l'argument «
[System.IO.Compression.ZipFileExtensions]::ExtractToFile».
Au caractère Ligne:1 : 107
+ ... argetPath = Join-Path $ExtractionDir $_.Name     [System.IO.Compressi ...
+                 ~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~~
    + CategoryInfo          : InvalidArgument : (:) [Join-Path], ParameterBindingException
    + FullyQualifiedErrorId : PositionalParameterNotFound,Microsoft.PowerShell.Commands.JoinPathCommand

PS C:\Users\HP\android-audit> Set-ExecutionPolicy -Scope CurrentUser -ExecutionPolicy RemoteSigned -Force
PS C:\Users\HP\android-audit> .\extraire_dex.ps1
Extrait : classes.dex
Termine !
PS C:\Users\HP\android-audit>
```

FIGURE 11 – Extraction du DEX et tentative de conversion avec *dex2jar*.

La capture montre aussi une erreur de commande au premier essai, puis une correction du script PowerShell. Après la correction, le traitement s'est terminé correctement. Cela m'a permis de vérifier que l'extraction du DEX était bien effectuée avant l'ouverture dans un autre décompilateur.

7 Comparaison entre JADX GUI et JD-GUI

J'ai comparé les deux outils sur le même type de contenu. JADX m'a paru plus pratique pour l'analyse Android, alors que JD-GUI reste utile pour parcourir rapidement le code Java décompilé.

Critère	JADX GUI	JD-GUI
Navigation	Très pratique pour Android : manifeste, ressources et classes sont accessibles au même endroit	Plus centré sur le code Java décompilé
Lecture du projet	Bonne vue d'ensemble de l'APK	Vue plus simple mais moins complète
Recherche	Recherche globale efficace	Recherche plus limitée selon les cas
Ressources Android	Oui, directement visibles	Non, ce n'est pas son point fort
Lisibilité	Bonne pour l'analyse mobile	Utile pour une lecture rapide du code

D'après mon expérience dans ce TP, j'utiliserais surtout JADX pour l'analyse principale d'un APK, puis JD-GUI en complément si je veux simplement relire un code déjà extrait.

8 Conclusion

Ce TP m'a permis de comprendre concrètement comment analyser statiquement une application Android. J'ai pu vérifier la structure de l'APK, identifier les informations importantes dans le manifeste, repérer plusieurs chaînes sensibles dans le code et comparer deux outils de décompilation.

L'analyse montre aussi qu'une application peut laisser apparaître des éléments sensibles comme des secrets, des messages de débogage ou des traces de journalisation. Pour cette raison, une application destinée à être distribuée doit éviter de conserver ce type d'information dans son code.

9 Annexe – Synthèse rapide

Élément	Résumé
APK analysé	*UnCrackable Level 1*
Type d'étude	Analyse statique
Outils	JADX GUI, dex2jar, JD-GUI
Constats principaux	Secret en clair, messages de débogage, logs techniques, vérification de test-keys, backup activé